

队伍编号	MCB2502562
赛道	(B)

物流理赔风险识别及服务升级问题

摘要

服务质量与用户体验是物流链中必不可少的一部分。面对增量巨大的物流业务，现代物流企业的理赔服务的“售前”环节通过分析历史数据识别货物揽收前的潜在风险，优化物流服务方案，从源头上降低异常事件的发生概率。本文围绕物流企业理赔服务“售前”环节的风险识别与服务升级问题，解决两个问题并构建相应模型。

针对问题一，本文先通过先清洗附件 1 中的原始数据，对异常值和缺失值进行处理。接着使用 Haversine+K-distance+DBSCAN 的三阶段地理聚类方法进行分析，运用 Haversine 公式将距离转化为弧度，以 K-distance 方法确定领域半径(ϵ)，通过 DBSCAN, K-Means 等聚类方法对处理后的数据进行聚类分析，并对不同方法的结果进行比较。然后计算每个区域的异常运单比例，按照比例识别重点风险区域。模型训练完成后，对优化后的模型进行评估，保留效果最优的聚类模型。结果表明，该方法有效识别出多个高风险聚集区，且聚类质量优异。

针对问题二，本文先通过特征工程对附件 1 和附件 2 中的数据进行处理，避免原始数据带偏预测模型。将附件 1 中的数据作为训练数据，附件 2 中的数据作为测试数据。接着采用 Stacking 集成学习法，以 LightGBM、XGBoost、CatBoost 三个模型为基模型进行训练和预测，用五折交叉验证获取基模型的 OOF 预测和测试集预测，并计算 OOF 的 AUC 评估性能。最终通过 LogisticRegression 元模型集成优化，AUC 值显著优于单一模型表现。

关键词：DBSCAN 聚类；异常检测；回归预测；风险识别与预测

目录

一、问题重述	1
1.1 背景知识	1
1.2 问题描述	1
二、问题分析	1
2.1 问题一分析	1
2.2 问题二分析	2
三、模型假设	2
四、符号说明	2
五、问题一模型的建立与求解	3
5.1 数据预处理	3
5.2 基于 Haversine+K-distance 的 DBSCAN 聚类分析	4
5.3 K-Means 聚类方法	9
六、问题二模型的建立与求解	12
6.1 特征工程	12
6.2 基模型训练	13
6.3 交叉验证与 OOF 预测	16
6.4 Stacking 集成	17
6.5 CatBoost+贝叶斯寻优法	19
七、模型评价与推广	20
7.1 模型的优点	20
7.2 模型的缺点	20
7.3 模型的改进与推广	20
八、参考文献	21
九、附录	21

一、问题重述

1.1 背景知识

随着物流行业的快速扩张，服务质量与用户体验成为物流链中必不可少的一部分。面对增量巨大的物流业务，物品丢失甚至破损的情况会损坏消费者的合法权益，对此进行的理赔服务成为当前物流企业运营的关键环节。现代物流企业的理赔服务可以分为“售前”和“售后”两个环节。售前环节的核心目标是主动预防，通过分析历史数据识别货物揽收前的潜在风险，从而优化物流服务方案，从源头上降低异常事件的发生概率。售后环节则侧重于在异常发生后进行合理高效的赔付，在保障客户权益的同时避免超额赔付，控制成本。

对于售前环节的风险识别与服务优化，物流企业可以通过对历史运单的数据深入挖掘，构建智能化的风险预警体系。一方面，基于地理信息可以将服务区域进行合理划分，识别出异常运单高发的“重点风险区域”。对于这些区域的运单，企业可以在揽收时采取加强包装、重点沟通等差异化服务措施，以防范风险。另一方面，在单个运单维度上构建预测模型，精准识别出高风险运单，对于这些运单在运输过程中给予重点监护，也能够降低理赔风险。因此，充分分析历史的异常订单数据，可以使物流企业在货物揽收时提升物流服务质量，更好提高相关用户的体验。

1.2 问题描述

问题 1：首先根据附件 1 中给出的运单经纬度，采用“POI”（地理兴趣点）的算法，将位置相近的地址聚合为多个区域。接着计算每个局和区域的异常运单比例，按照比例来识别重点风险区域，并将地址划分及风险识别结果及对应异常率填在结果表 1 中。

问题 2：首先分析附件 1 中异常运单的通用特征规律，通过该特征规律来建立异常运单识别模型。接着利用该模型对附件 2 中待识别的运单进行预测，判断其是否为潜在的异常运单。最后根据识别结果与保障服务运单的上限，说明附件 2 中适用于该服务的运单，并将结果填入结果表 2。

二、问题分析

2.1 问题一分析

问题一要求根据附件一中给出的运单经纬度，用 POI 算法将位置相近的地址聚合为多个区域，并根据每一类里异常运单所占比例识别重点风险区域。要先清洗附件 1 中的原始数据，对异常值和缺失值进行处理，进行描述性分析。接

着运用 Haversine 公式将距离转化为弧度,以 K-distance 方法确定领域半径(ϵ),通过 DBSCAN, K-Means 等聚类方法对处理后的数据进行聚类分析,并对不同方法的结果进行比较。然后计算每个区域的异常运单比例,按照比例识别重点风险区域。模型训练完成后,对优化后的模型进行评估,保留效果最优的聚类模型。

2.2 问题二分析

问题二要求分析附件 1 中异常运单的通用特征,通过该特征规律来建立异常运单识别模型,并对附件 2 中待识别的运单进行异常识别。根据识别结果说明附件 2 中适合推荐该服务的运单。要先通过特征工程对附件 1 和附件 2 中的数据进行预处理,避免原始数据带偏预测模型。将附件 1 中的数据作为训练数据,附件 2 中的数据作为测试数据。接着采用 Stacking 集成学习法,以 LightGBM、XGBoost、CatBoost 三个模型为基模型进行训练和预测,得出结果后,用五折交叉验证获取基模型的 OOF 预测和测试集预测,并计算 OOF 的 AUC 评估性能。最后使用 Stacking 集成法得到集成模型的最终预测概率。

三、模型假设

- 本文将作如下假设、便于模型的建立与求解;
- 1.假设附件所提供的历史运单数据均为真实业务场景下产生的有效数据;
 - 2.假设对于数据中缺失值的处理方式不会对预测结果造成太大误差;
 - 3.假设对数据中缺失值、异常值的处理方式不会对模型的整体预测结果造成显著偏差,能够保持数据的主要统计特性;
 - 4..假设历史数据中所揭示的异常运单特征规律在未来一段时间内将继续存在,使得基于历史数据训练的预测模型对未来的新运单具有有效的预测能力。

四、符号说明

符号	含义与说明
φ, λ	分别表示纬度和经度,单位为弧度
X_{meta}	由基模型的 OOF 预测构成的元特征矩阵
$N_{\epsilon}(p)$	点 p 的 ϵ 邻域内所有点的集合
$\epsilon(\epsilon)$	邻域半径参数
$minPts$	形成核心点所需的最小邻域点数
$kdist(p_i)$	点 p_i 到其第 k 个最近邻的距离
$\hat{y}_i^{(j)}$	第 j 个基模型对第 i 个样本的预测值

五、问题一模型的建立与求解

5.1 数据预处理

通过对数据的查看，可以发现部分数据存在异常，且有些数据缺失，此类情况不仅可能影响后续的分析过程,还可能影响最终结果的准确性。为确保数据质量并提升分析准确性和可靠性，要对数据进行预处理。

（1）重复值的处理

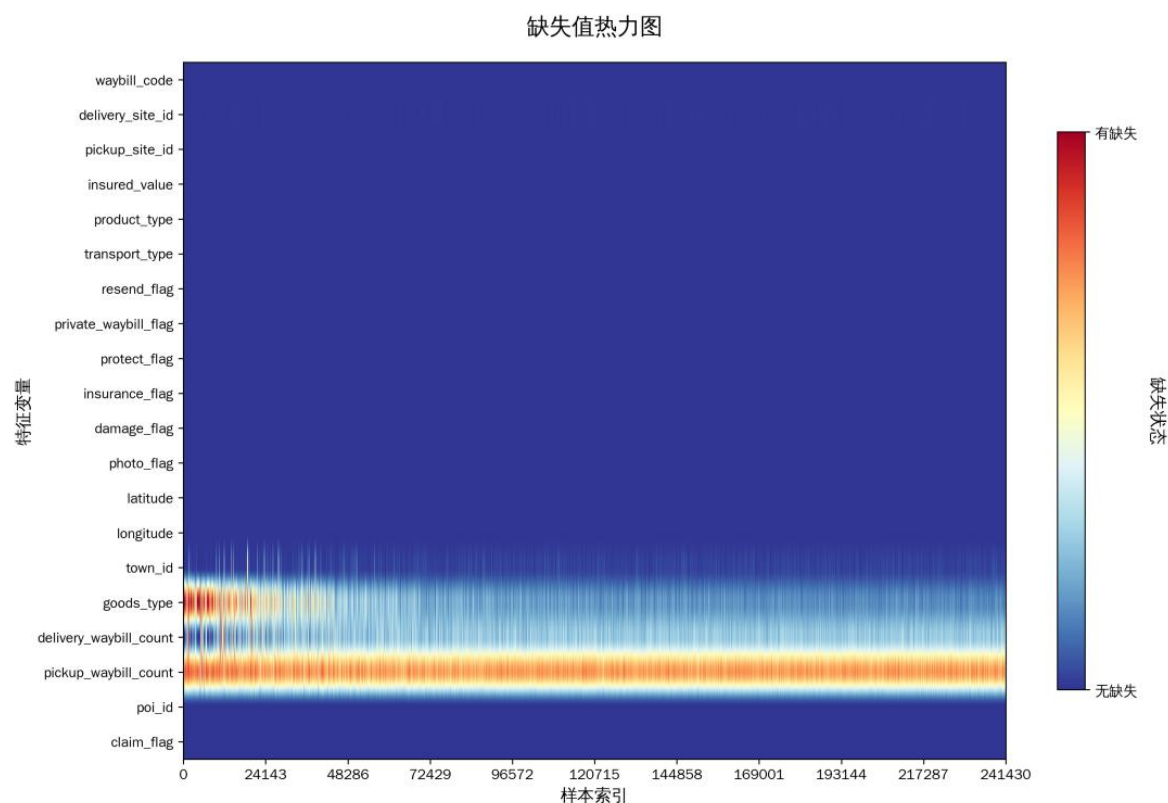
通过根据附件 1 中国际快递单号数据筛选重复订单，未发现重复值，保障了数据分析的准确性和机器学习模型的可靠性。

（2）异常数据的修正

通过分析附件 1 与附件 2 的数据，本文删除了货物类型、配送运单数和取件运单数三个变量。货物类型、配送运单数和取件运单数三列与其他变量关系较弱，且未知数据较多。为了保证模型的稳健性与准确性，本文认为这几项数据应当直接删除。

（3）缺失数据的处理

通过对附件 1 与附件 2 数据的筛选，发现缺失值主要存在于“街道 ID”与“配送站点 ID”两列，为保证样本的可识别性，本文将两列有缺失值的数据条直接删除，共删除 6583 条。



5.2 基于 Haversine+K-distance 的 DBSCAN 聚类分析

5.2.1 运用 Haversine 公式将距离转化为弧度

Haversine 公式是一种用于计算地球上两点之间距离的方法。由于经纬度是球面坐标，而标准欧氏距离基于平面假设，如果强行使用欧氏距离会导致地理距离的扭曲。

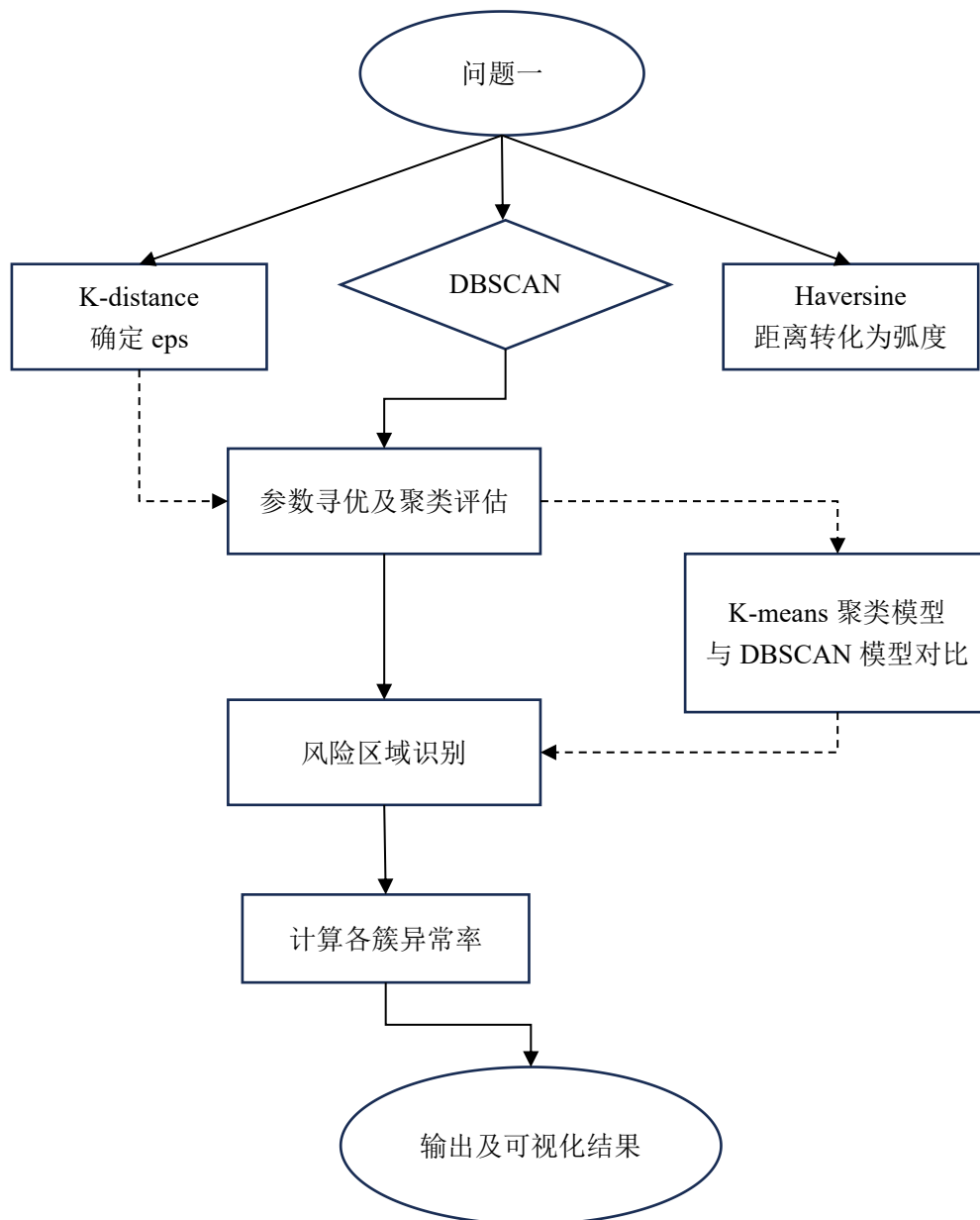


图 2 问题一思路图

假设 A 和 B 的经纬度分别为：

A: (φ_1, λ_1) B: (φ_2, λ_2)

其中 φ 表示纬度， λ 表示经度，单位为弧度。

球体半径为 R （通常取地球平均半径 6371km），则两点之间的大圆距离 d 为：

$$d = R \cdot c$$

其中

$$a = \sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)$$

$$c = 2 \cdot \arctan\left(\sqrt{\frac{a}{1-a}}\right)$$

为了准确计算球面距离，本文将经纬度转化为弧度，运用 Haversine 公式作为 DBSCAN 的 *metric*，假设地球是一个理想的球体，基于两点的经度和纬度，计算地球表面两点间的大圆距离，确保聚类反映真实地理。

5.2.2 以 K-distance 方法确定领域半径 (eps)

K-distance 方法是一种启发式、可视化的参数选择技术，用于为基于密度的聚类算法自动确定领域半径 (eps) 参数。 eps 和 *min sample size* 最优值的确定具有一定的不确定性，且本题中数据量庞大，使用网格法或贝叶斯方法寻优难以实现。同时，手动尝试也难以掌握全局最优，效率低下。因此，本文采用 K-distance 方法分析数据集中每个点到其第 k 个最邻近的距离分布，找到能区分密集区域和稀疏区域的拐点距离值，作为 DBSCAN 的 eps 参数。

给定数据集 D 和正整数 k （通常 $k = minPts$ ）首先为每个点 $p \in D$ 计算其到第 k 个最近邻的距离：

$$kdist(p_i) = d(p_i, k - th \text{ nearest neighbor of } p_i)$$

其中， d 是距离度量， k 通常取数据维度的 2 倍。接着将所有的 $kdist$ 值从大到小排序，绘制 K-distance 图。曲线上最显著的“拐点”对应的距离值，即为最推荐的 eps 参数。本文采用 k-distance 方法寻找到曲率最大点 (*elbow*) 作为 eps 起点，确定优秀的 eps 值的大致范围，对全局进行掌握，再通过手动尝试的方法找到最优的 eps 和 *min sample size* 的大小。

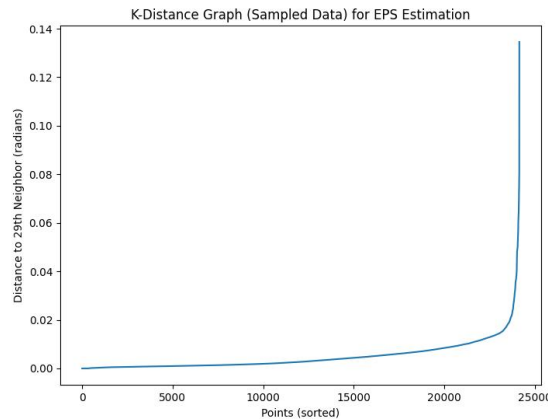


图 3 K-distance 图

如图所示，曲率最大点(*elbow*)给出了 *eps* 上界，即 0.01-0.02 rad~63-127km，表示超过此距离视为噪声。但本题中的“连通区域”应当为小规模簇，如果直接使用 0.01 rad *eps*，则会形成过大簇。因此，基于此上限，我们从小值 0.0005-0.003 rad ~3-18km 开始尝试。

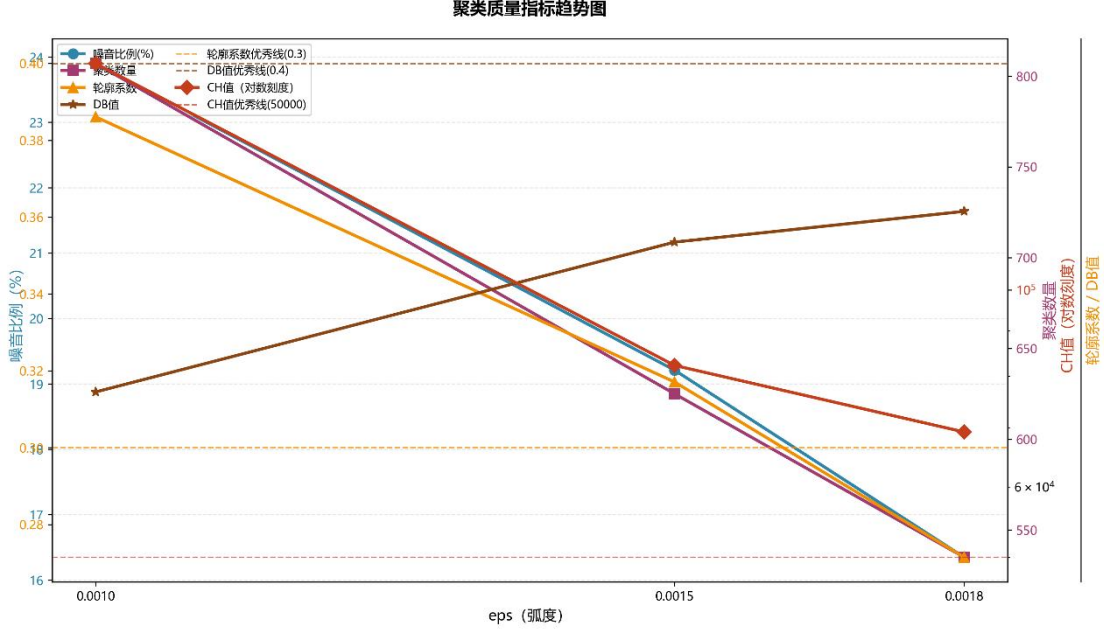


图 4 聚类质量指标趋势图

根据 DBSCAN 聚类的参数调优逻辑，本文以 *eps* 为核心调整维度，得出聚类质量指标趋势图。图 n 清晰地呈现了 *eps* 增大过程中 5 类核心评估指标的变化规律。图中噪音比例从 23.90% 降至 16.35%，聚类数量从 807 降至 535。随着 *eps* 的增大，更多点被纳入簇、噪音减少、簇数量合并，验证了参数调整的合理性。同时，轮廓系数从 0.3861 降至 0.2715，CH 值从 179984 降至 69190 (远高于 50000 的优秀线)，DB 值从 0.3145 升至 0.3615 (小于 0.4 的优秀线)，可以看出三种模型的聚类质量均处于优秀区间。

5.2.3 DBSCAN 聚类分析

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种基于密度的空间聚类算法。它通过检测数据空间中高密度区域，并将这些区域划分为簇，同时识别并排除低密度区域的噪声点。DBSCAN 有两个关键参数：领域半径 ϵ (*eps*) 和形成稠密区域所需的最小点数 *minPts*。点 *p* 的 ϵ 邻域定义为：

$$N_{\epsilon}(p) = \{q \in D | dist(p, q) \leq \epsilon\}$$

其中，*dist* 是距离函数。如果 $|N_{\epsilon}(p)| \geq minPts$ ，则为核心点；若不是核心点，但落在某个核心点的 ϵ 邻域内，则为边界点；既不属于核心点也不属于边界点，则为噪声点。对每个未访问点 *p*，若 $|N_{\epsilon}(p)| < minPts$ ，则标记 *p* 为噪声；否则创建新簇，递归拓展所有密度可达点，拓展时将边界点分配给第一个到达它的核心点所在簇。

DBSCAN 将数据空间划分为：

$$\text{数据空间} = \text{高密度区域（簇）} \cup \text{低密度区域（噪声）}$$

核心公式为：

$$\text{Cluster} = \{x \mid \exists \text{ 核心点 } c : x \text{ 从 } c \text{ 密度可达}\}$$

$$\text{Noise} = D \cup \text{Cluster}$$

本文使用 DBSCAN 方法，按照 K-distance 中确定的 *eps* 和 *min sample size* 的大概范围，将经纬度相近的 POI 聚合成类，形成连通区域，计算每个簇的异常率来识别风险区域，得到如表所示的三个结果。

	Method 1	Method 2	Method3
Noise percentage	23.91%	19.21%	16.35%
Number of clusters	807	625	535
Silhouette	0.2861	0.3171	0.2715
CH	179984.6032	82230.6063	69190.3210
DB	0.3145	0.3535	0.3615

表 1 聚类结果评估表

我们采用五种指标全面衡量三种参数的拟合程度：

- （1）噪音比例表示没有被覆盖到的样本比例；
- （2）聚类组数表示将全部样本点分成的簇的个数，聚类组数应该在一个合理的范围内，若数量过多则会是的快递公司的现实管理难度大幅上升，而数量过少则会无法精确识别 POI；
- （3）Silhouette 系数表示每个数据点与其自身簇的相似度（凝聚）相对于其他簇的分离度；该系数评估簇的内部紧致性和外部分离性，越靠近 1 越好，在地理类研究中靠近 0.3 为较好数据；
- （4）CH 值衡量簇间分散度与簇内分散度的比率。该系数评估整体聚类的分离性和紧致性，值越大越好；
- （5）DB 值衡量每个簇与其最相似簇的平均相似度。该系数评估簇的内部分散与簇间距离的比率，越小越好。

由于 V3 的噪音比例较高，且 Silhouette 系数分离度较低，说明簇数分离度不足，最终结果偏离度可能较高，V5 和 V6 数据的吻合程度更高，结构更加清晰，因此本文选择对 V5 和 V6 进行后续分析。

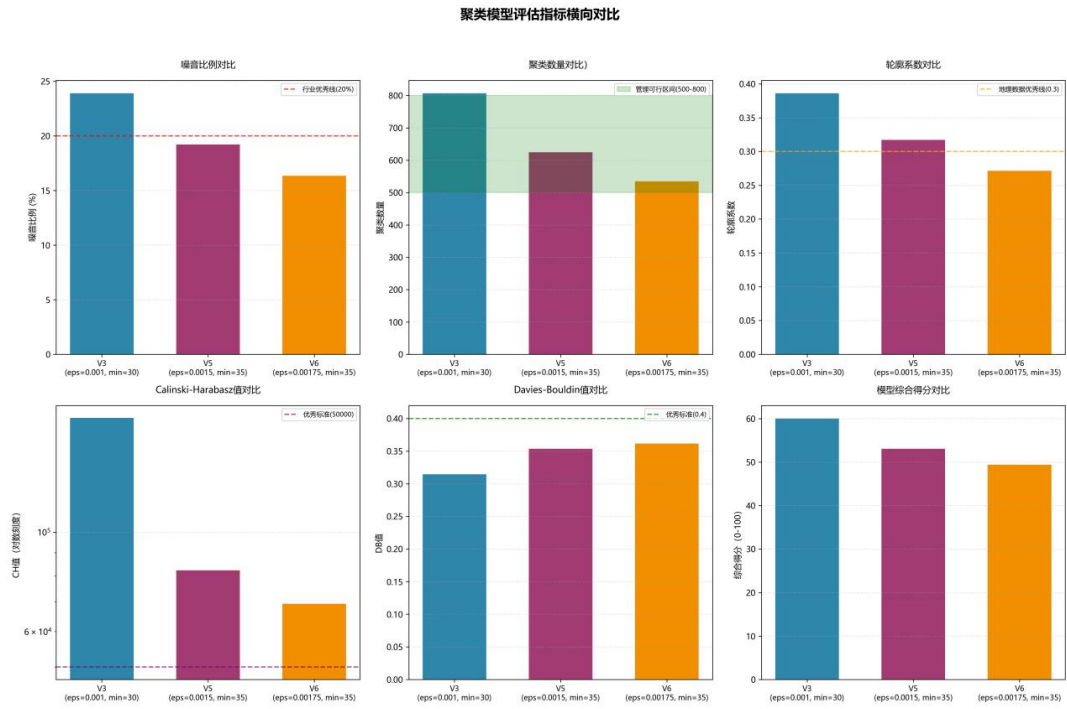


图 5 聚类模型评估指标横向对比图

如图 5 所示，三种参数下噪音比例偏低，Silhouette 系数靠近 0.3，CH 值均>50000，DB 值都处于较小的范围内，表示三种参数的聚类效果都很好。图 6 和图 7 呈现了 V5 和 V6 的可视化结果。

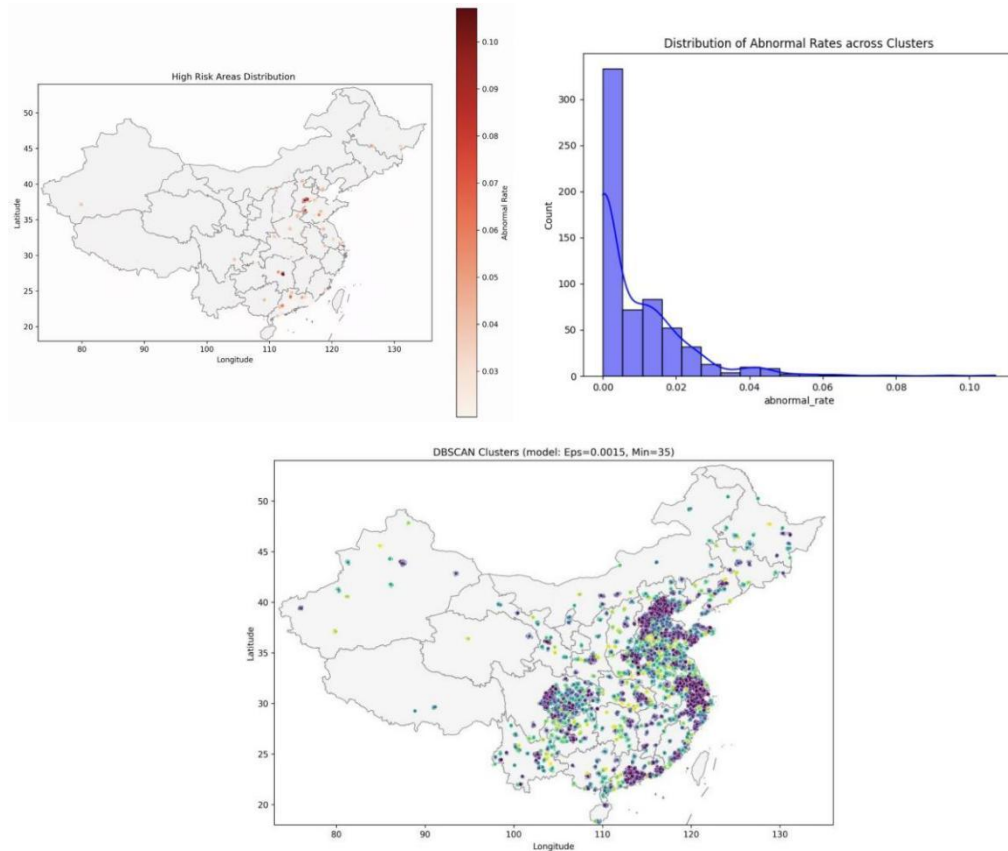


图 6 V5 可视化结果

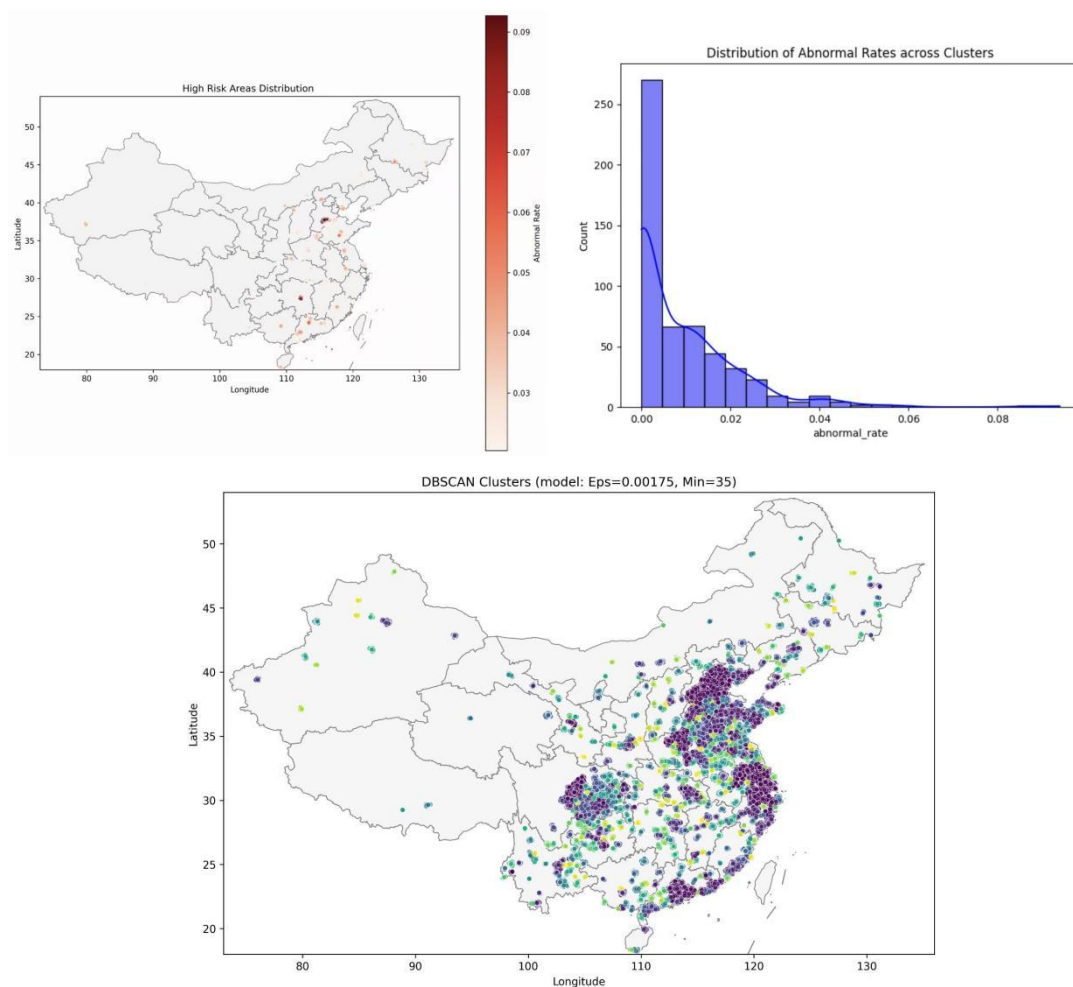


图 7 V6 可视化结果

5.3K-Means 聚类方法

K-Means 方法的核心目标是将给定的数据集划分成 K 个簇(K 是超参), 并给出每个样本数据对应的中心点。K-Means 方法通过迭代计算样本点与簇中心的距离来实现聚类,这种线性复杂度使得算法在处理大规模地理位置数据时具有明显优势, 能够快速完成聚类任务。

如表 2 所示, K-Means 方法将数据分成了 15 个簇, 轮廓系数为 0.4753, CH 值非常高, DB 值为 0.6908, 说明在数据上能形成有一定区分度的聚类, 但约 21% 的点是噪声。

表 2 K-means 聚类结果评估

评估指标	数值
Noise percentage	20.87%
Number of clusters	15
Silhouette	0.4753
CH	300159.58
DB	0.6908

肘部寻优法是一种用于确定聚类算法（特别是 K-Means）最佳聚类数 K 的启发式方法。在初始阶段，增加 K 会带来 SSE 的显著下降，这意味着聚类结构被有效揭示；但超过某个点后，再增加 K 带来的 SSE 下降会变得微乎其微，这意味着继续细分已不能获得有意义的聚类结构提升。肘部寻优法通过直观、可视化的手段，依据数据本身的聚拢特性，选择一个在“模型复杂度”和“聚类效果”之间取得平衡的 K 值。

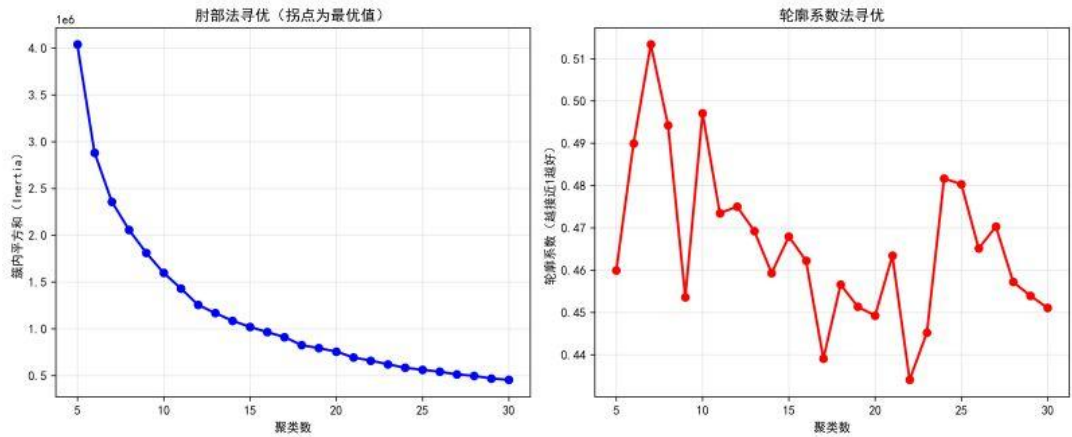


图 8 肘部寻优法与轮廓系数法寻优

轮廓系数法则是从另一个角度评估聚类质量的方法，对于一个样本集合，它的轮廓系数是所有样本轮廓系数的平均值。轮廓系数的取值范围是 $[-1, 1]$ ，同类样本距离越相近不同类别样本距离越远，分数越高，聚类效果越好。DBI 指标（Davies-bouldin）用来衡量任意两个簇的簇内距离之后与簇间距离之比。该指标越小表示聚类效果越好。CH（Calinski-Harbasz Score）指标通过计算类内各点与类中心的距离平方和来度量类内的紧密度（分母），通过计算类间中心点与数据集中心点距离平方和来度量数据集的分离度（分子），CH 指标由分离度与紧密度的比值得到，CH 越大表示聚类效果越好。

本文使用肘部寻优法确定 K-Means 的最优簇数，聚类数 15 是曲线的“拐点”，既保证了簇内样本的紧凑性，又避免了聚类数过多导致的冗余，是最优聚类数。

从图 8 中可以看出当聚类数从 5 增加到 15 左右时，簇内平方和快速下降（曲线斜率很陡），说明增加聚类数能显著提升簇内紧凑性；当聚类数超过 15 后，簇内平方和的下降速度明显变缓，说明继续增加聚类数，簇内紧凑性的提升幅度很小。



图 9 K-means 聚类结果

如图 9 所示，高风险点有明显的地理聚集，且集中在特定片区，并非随机分布，说明风险确实有地理相关性，21%的噪声点被强制分配到某个簇，污染了聚类纯度，使得结果可能不符合真实地理分布。在本题中，对比 K-Means 方法，DBSCAN 方法更加合适。

六、问题二模型的建立与求解

6.1 特征工程

特征工程是指利用数据领域的相关知识，对原始数据进行处理、转换、组合，构建出更适合机器学习模型使用的特征变量。附件 1 与附件 2 中的数据存在原始问题：数据有缺失，如“派送运单数”“揽收运单数”，需要进行缺失值处理；经纬度缺失，会增加噪声；变量之间存在潜在的非线性关系，需要进行组合变换。

如图所示，大多数物流变量之间相关性较弱，“运输方式”与“补发”“私单”等变量有中等正相关关系，说明某些特定运输方式常伴随补发或私密运单业务。“保价”与“保险”之间有一定正相关关系，说明用户选择保价时也更可能同时购买保险。总体来说这些变量在数据之中相对独立，共线性问题较弱。

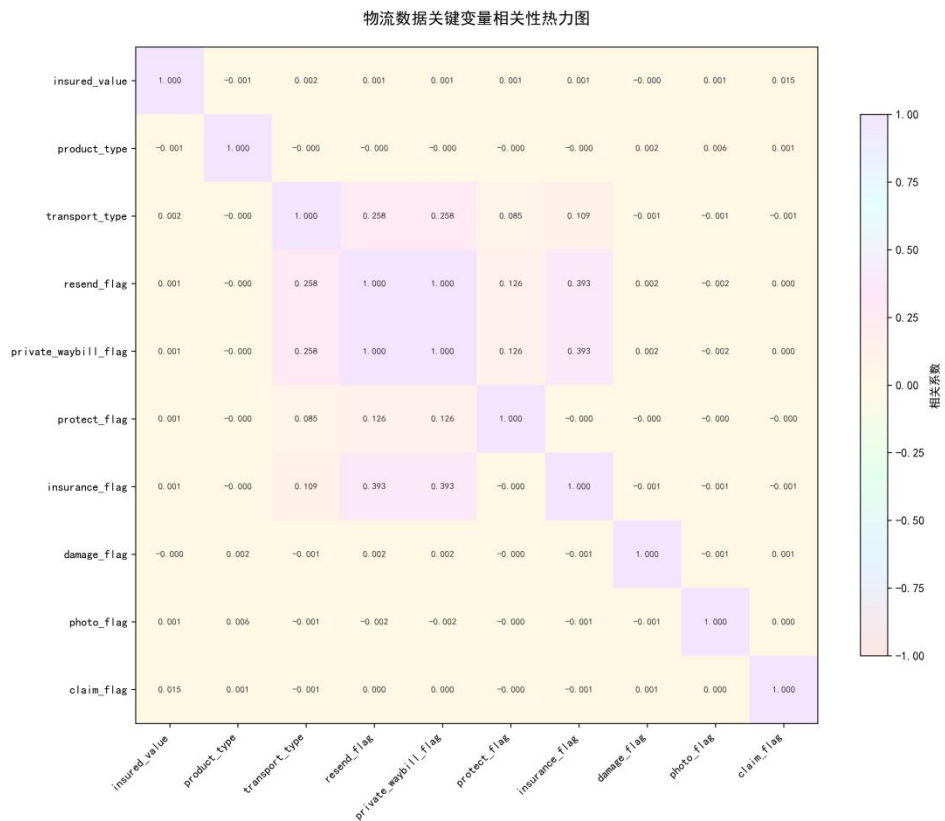


图 10 物流数据关键变量相关性热力图

(1) 处理异常值、缺失值

通过对附件 1 和附件 2 数据的分析，本文删除了“运单号”这一变量，发现缺失值主要存在于“派送运单数”“揽收运单数”“区域编码”“货物类型”“派送站点 ID”“揽收站点 ID”。“运单号”这一变量只适用于编码不同的个体，且杂乱无序，没有其他实际意义，为提高模型准确性，应当直接删除。

对于数值变量，如“派送运单数”和“揽收运单数”，本文使用均值填充缺失值，保持单量指标的统计意义，不影响模型对正常业务量的判断。对于分类变

量，如“区域编码”“货物类型”“派送站点 ID”“揽收站点 ID”，本文用 *unknown* 填充，并将其转化为字符串格式，以适配 CatBoost 的特征处理。

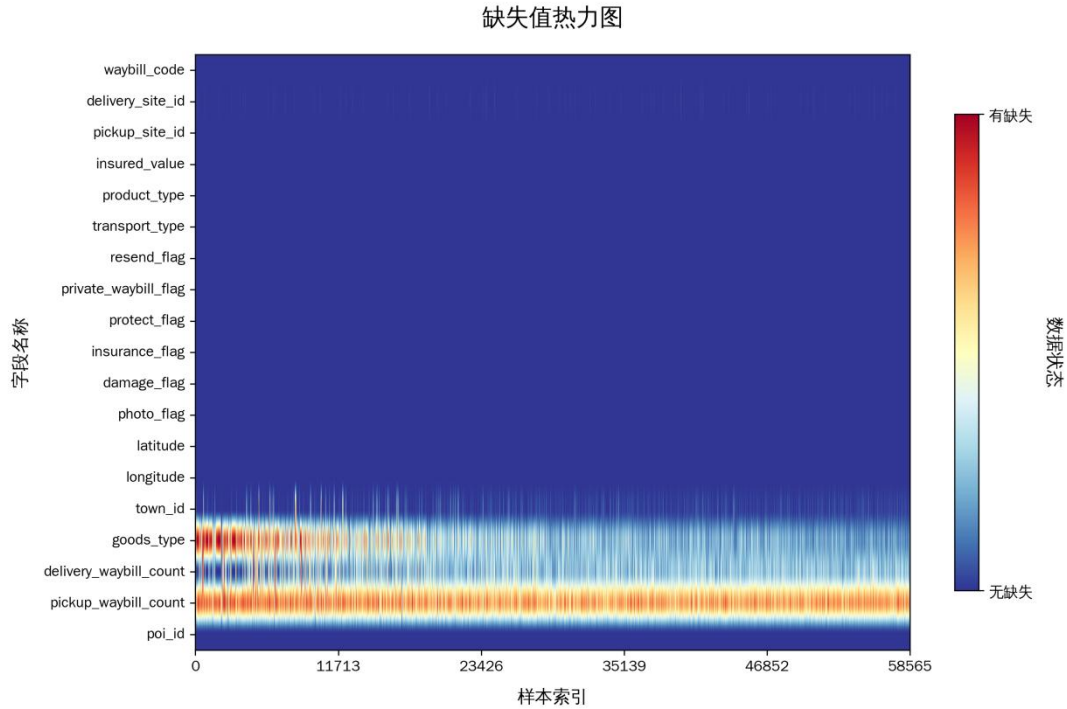


图 11 附件 2 缺失值热力图

（3）经纬度处理

本文将缺失的经纬度用-999 填充，并创建缺失标记特征（*lat_missing*），用极端值进行标记，保障模型不会忽略缺失。同时，由于经纬度是连续值，两个相距 1 米的点视为“不同”，但业务上可能属于同一片区。因此本文对经纬度进行分箱处理，将经纬度分为 20 个区间，将这一连续变量分为不同的区域，增加业务意义，减少噪声。

（4）新增特征变量

本文通过交叉特征新生成“站点单量比率”变量。该比率能够更好地反映站点业务类型，能够更好地捕捉数据特征，增强对业务站特征的刻画。

$$\text{站点单量比率} = \frac{\text{揽收量}}{\text{派送量} + \varepsilon}$$

其中

$$\varepsilon = 1e - 5$$

6.2 基模型训练

为提高预测结果准确性，避免单模型数据效果不佳，本文采用集成学习法，以 LightGBM、XGBoost、CatBoost 三个模型为基模型进行训练和预测。得出结果后，用五折交叉验证获取基模型的 OOF 预测和测试集预测，并计算 OOF 的

AUC 评估性能。最后使用 Stacking 集成法得到集成模型的最终预测概率。

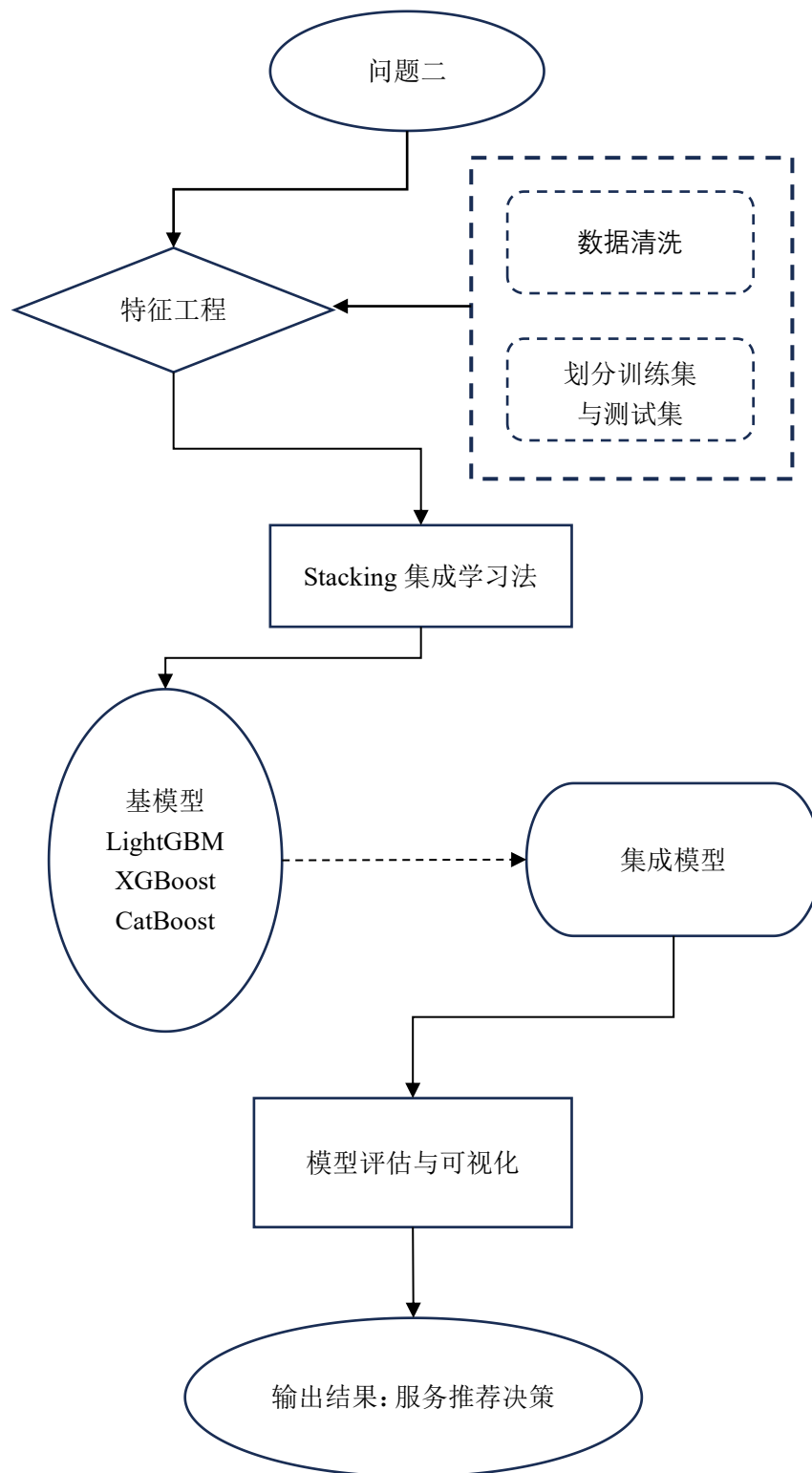


图 12 问题二思路图

6.2.1 LightGBM 算法

LightGBM 模型是基于梯度提升决策树（Gradient Boosting Decision Tree，

GBDT)的思想,通过迭代地训练多棵决策树,每棵树都试图学习之前树的残差,从而逐步优化模型的预测能力,实现了对一系列基于决策树的传统 Boosting 算法的改进。该模型通过使用直方图加速技术,将连续型特征离散成 k 个离散值,并构造 k 个直方图,使得 LightGBM 能够更高效地构建决策树,提升模型训练速度,适合处理大规模数据集和高维特征。步骤如下

1.分桶:将连续特征值离散化到固定数量的 bins 中(如 255 个)

2.构建直方图:基于 bins 来构建梯度直方图。对于一个节点,直方图统计了每个 bin 内样本的梯度之和(G)与样本数量(H)。

3.寻找最优分裂点:在直方图上遍历所有可能的分裂点,计算分裂增益。

分裂增益公式为:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

其中, G_L , H_L 为左子节点的梯度之和与样本数, G_R , H_R 为右子节点的梯度之和与样本数, λ 为 $L2$ 正则化项, 惩罚复杂的树结构, γ 为分裂所需的最小增益。

本文使用 LightGBM 预测风险标注,提高结果准确率,进行的多维度特征整合分析。LightGBM 通过复杂的特征进行交互建模,并捕捉各类特征之间的非线性关系,同时,通过调整超参数,避免过拟合,同时提高模型的泛化能力。

6.2.2 XGBoost 算法

XGBoost 是一个基于梯度提升树(GBDT)的机器学习算法,通过优化目标函数来训练模型,目标函数包含损失函数和正则化项两部分。损失函数衡量模型预测与实际标签之间的差异,正则化项则控制模型复杂度,避免过拟合。

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

其中, $\mathcal{L}(\theta)$ 为 XGBoost 的目标函数, $\ell(y_i, \hat{y}_i)$ 为损失函数,表示第 i 个样本的实际值和预测值之间的差异; $\Omega(f_k)$ 为正则化项,表示树模型的复杂度。

损失函数是衡量模型预测值与实际标签之间差异的指标。在 XGBoost 中,通常是用均方误差(MSE)作为损失函数,公式为:

$$\ell(y_i, \hat{y}_i) = \frac{1}{2} (y_i - \hat{y}_i)^2$$

损失函数通过计算实际标签 y_i 与预测值 $\hat{y}_i$ 之间的差异来衡量模型的表现。在回归问题中,常用的损失函数是均方误差(MSE),它能够量化模型预测值与实际值之间的平方差异。XGBoost 通过最小化损失函数来优化模型的预测能力。每次迭代中,模型通过计算损失函数的梯度来更新参数,减少预测误差。

正则化项用于控制模型的复杂度,防止过拟合。XGBoost 的正则化项包括两

个部分:树的复杂度（树的叶子数）和叶子节点的权重大小。具体的正则化表达式为:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

γT 是树的复杂度项，其中 T 是树的叶子节点数量， γ 是控制树复杂度的超参数。增加叶子节点数会增加模型的复杂度，可能导致过拟合，因此需要通过正则化来控制叶子节点的数量。 $\frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ 是对叶子节点权重的正则化项，其中 w_j 是第 j 个叶子节点的权重， λ 是正则化参数。这个项通过惩罚较大的叶子节点权重来减少模型的复杂度。

6.2.3 CatBoost 算法

CatBoost 是专注于处理类别性特征，是一种改进的 GBDT 算法，通过类别特征编码将分类特征转为数值特征，同时引入有序提升技术，通过随机排列训练集顺序减少过拟合。本文采用

CatBoost 继承了 GBDT 的加法模型和前向分步优化思想，每一轮迭代通过拟合前一轮的残差，不断提升整体预测能力。

加法模型表达:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in \mathcal{F}$$

其中， K 为树的数量， f_k 为第 k 棵决策树， $\mathcal{F}$ 为所有树的集合。

损失函数表达:

$$L(y, F(x)) = \frac{1}{n} \sum_{i=1}^n l(y_i, F(x_i))$$

每一轮的模型更新为:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

$h_m(x)$ 为当前轮新生成的树， γ_m 为步长。

CatBoost 最大创新之一是有序 Boosting (Ordered Boosting)，通过多次打乱数据顺序、仅用“历史”样本估算目标统计量，极大减少目标泄露和预测偏移，降低过拟合风险。

6.3 交叉验证与 OOF 预测

五折交叉验证 (5-Fold Cross-Validation) 是机器学习中常用的模型评估方法。它通过将数据集分成 5 个互斥的子集，轮流用其中 4 折训练模型，剩余 1 折验证模型性能，最终取 5 次验证结果的平均值作为模型的泛化能力评估。本文通过这种方法避免模型过拟合或欠拟合，减少因单一验证集划分导致的结果偏差，更客观地反映模型在未知数据上的性能。该方法无需单独划分验证集，让所有数据都参与训练和验证，尤其适合小数据集场景；5 次结果的波动程度可反映模型的稳

定性（波动越小，模型越稳定）。

OOF 预测是指在使用 K 折交叉验证训练模型时，每一折的模型在其对应的验证集上做出的预测。最终将所有折的验证集预测拼接起来，形成一个与原始训练集完全等长、一一对应的预测向量。该方法能够完整覆盖每个样本，分布近似于模型在真正独立测试集上的预测分布，提供了比单一训练-测试分割更可靠的模型性能估计。

6.4 Stacking 集成

（1）元特征构建

元特征是 Stacking 集成中的中间特征，由基模型在交叉验证过程中生成的 OOF 预测构成。每个基模型为每个训练样本生成一个预测值，这些预测值组合成新的特征矩阵。本文将 LightGBM、XGBoost 和 CatBoost 三个基模型各自产生的 OOF 预测拼接为元训练集（*meta_train*），将测试集的平均预测拼接为元测试集（*meta_test*），将不同模型的预测视角融合成一个统一的特征表示。

假设有 M 个基模型，训练集有 N 个样本：

$$\text{元特征矩阵 } X_{\text{meta}} = \begin{bmatrix} \hat{y}_1^{(1)} & \hat{y}_1^{(2)} & \cdots & \hat{y}_1^{(M)} \\ \hat{y}_2^{(1)} & \hat{y}_2^{(2)} & \cdots & \hat{y}_2^{(M)} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{y}_N^{(1)} & \hat{y}_N^{(2)} & \cdots & \hat{y}_N^{(M)} \end{bmatrix}$$

其中 $\hat{y}_i^{(j)}$ 表示第 j 个模型对第 i 个样本的 OOF 预测。

（2）元模型训练

元模型是在元特征矩阵上训练的模型，它不直接看原始特征，只基于基模型的“投票”来做最终决策。本文使用 LogisticRegression 作为元模型，输入元训练集，以原始标签 *claim_flag* 为目标进行训练，学习基模型预测结果与真实标签的关系，用于训练最终的 Stacking 元模型，最小化集成预测与真实标签的差异。对于线性元模型：

$$\hat{y}_{\text{ensemble}} = \sigma \left(w_0 + \sum_{j=1}^M w_j \cdot \hat{y}^{(j)} \right)$$

其中 w_j 是元模型学到的权重，反映第 j 个模型的重要程度， σ 是 *sigmoid* 函数。

（3）集成预测

集成预测是使用完整 Stacking 方法对新数据进行预测的过程，分为两步：先用所有基模型预测，再用元模型组合这些预测。元模型对元训练集和元测试集进行预测，得到集成模型的最终预测概率并进行可视化，降低对单个模型失效的敏感性。

表 4 预测模型 AUC 对比	
模型	AUC
LightGBM	0.6812
XGBoost	0.7015
CatBoost	0.7442
Stacking 集成模型	0.7465

如图 13 所示，集成模型的精确率-召回率曲线整体较平缓，且在召回率较低时精确率下降很快，说明正常样本远多于风险样本，属于类别不平衡数据。混淆矩阵中正确预测的模型占大多数，说明模型十分有效。

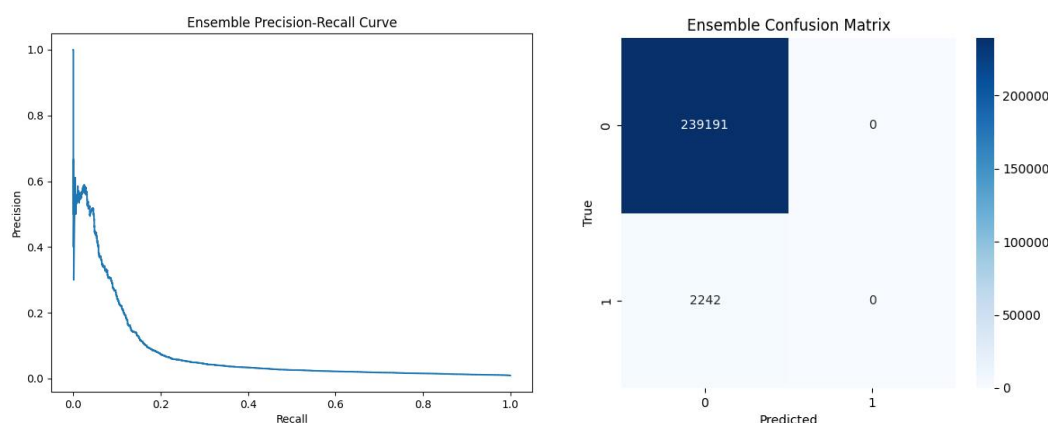


图 13 精确率-召回率曲线与混淆矩阵

本文同时分析了 CatBoost 特征重要性，如图 14 所示，“经度”和“纬度”时最重要特征，说明地理位置时预测风险的最强信号，高风险时间具有显著的空间聚集性。“店铺”“货物类型”“派送价值”等特征也与具体业务实体和货物属性有较强相关性。

从 ROC 曲线对比可以得出最终集成模型表现最佳，AUC 高达 0.9842，泛化能力较强，预测结果十分可靠。

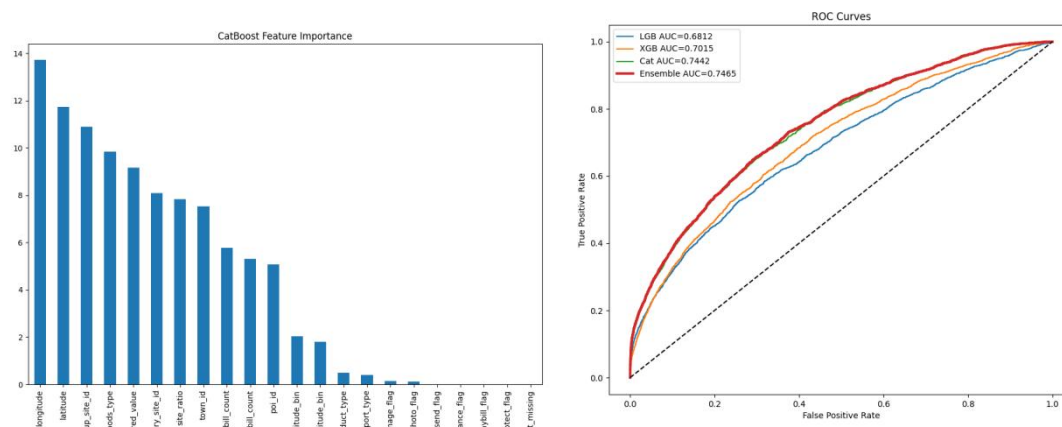


图 14 特征重要性统计与 ROC 曲线

6.5 CatBoost+贝叶斯寻优法

为提高模型准确性，本文还使用了 CatBoost+贝叶斯寻优法.贝叶斯寻优是一种高效的超参数调优方法，核心目标是找到让模型在验证集上 AUC 最大化的最优参数组合。贝叶斯优化通过概率模型学习参数与模型性能的关系，定向选择更可能最优的参数，减少调参次数，尤其适合 CatBoost 这类训练成本较高的模型。本文使用贝叶斯寻优进行了 40 次参数寻优，并且得到了最大化 AUC 的 Catboost 模型各项特征，如表所示。

表 5 CatBoost 模型各项特征

<i>iterations</i>	914
<i>depth</i>	7
<i>learning_rate</i>	0.0138
<i>l2_leaf_reg</i>	9.202
<i>random_strength</i>	0.0914
<i>bagging_temperature</i>	0.628
<i>border_count</i>	127

在得到最优参数后，本文进行进一步的模型训练和预测。首先将训练集分为特征（X）和目标变量（y），并按 4：1 的比例拆分训练集和验证集。接着进行类别不平衡处理，按照以下公式计算权重，用于模型训练时平衡类别。

$$scale_pos_weight = \frac{未理赔样本数}{理赔样本数}$$

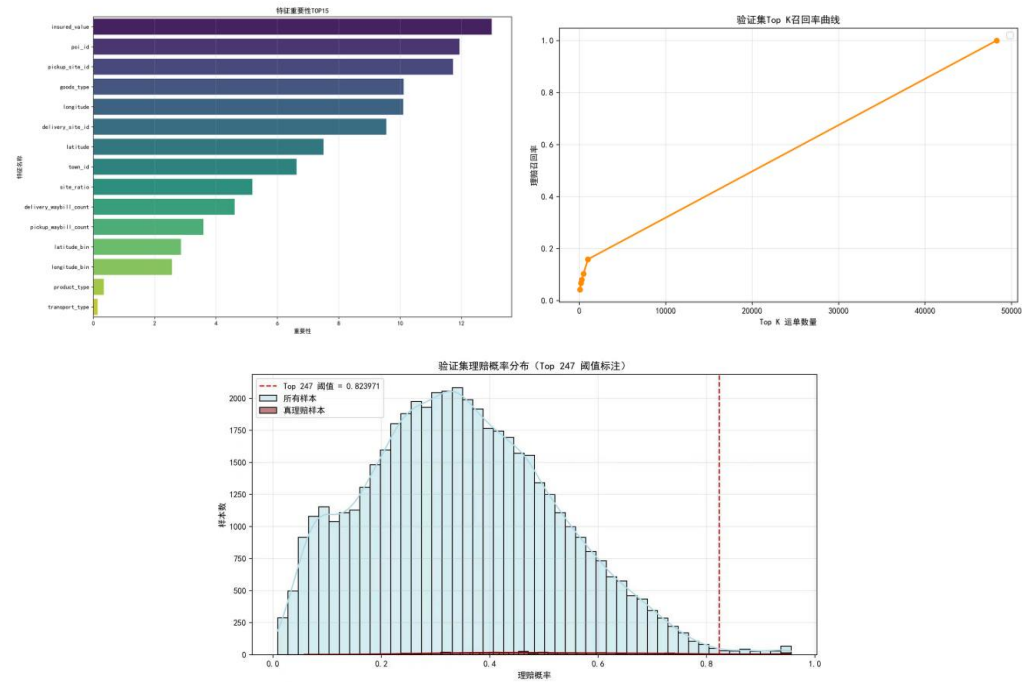


图 15 CatBoost+贝叶斯寻优方法结果可视化

最后通过 Pool 对象封装训练集和验证集,拟合模型并得到如图 15 所示结果。该模型的 AUC 为 0.74, 精确率为 13.36%, 召回率为 7.37%, 各项指标较好, 但准确率显著低于 Stacking 集成模型, 因此最终选择 Stacking 集成模型。

七、模型评价与推广

7.1 模型的优点

1.DBSCAN 聚类算法是一种基于密度的无监督聚类方法,自动发现任意形状的簇并明确分离噪声, 适合找地理聚集区。

2.LightGBM 模型基于梯度提升决策树的思想,通过迭代地训练多棵决策树,每棵树都试图学习之前树的残差,从而逐步优化模型的预测能力,实现了对一系列基于决策树的传统 Boosting 算法的改进。

3.XGBoost 回归预测模型具有高度正确性,和对异常值和噪声数据具有较高的鲁棒性,可以很好的解决实际问题。

4.CatBoost 通过多次打乱数据顺序、仅用“历史”样本估算目标统计量,极大减少了目标泄露和预测偏移,降低过拟合风险。

5.Stacking 集成学习法通过多个模型交叉验证降低了漏报率,降低过拟合风险。

7.2 模型的缺点

1.XGBoost 模型比较复杂, 参数过多, 需要进行调整参数来获得最佳效果。

2..K-Means 聚类算法对初始中心点的选择较为敏感,不同的初始点可能会得到不同的聚类结果。

3.CatBoost 模型训练速度较慢,本题为大数据集,调参成本高,训练时间长。

7.3 模型的改进与推广

尽管本文构建的模型体系在物流风险区域识别与异常运单预测上取得了良好效果,但仍存在可优化的空间。面对物流数据中可能存在的更多维度特征,可引入降维技术或深度学习特征提取方法,以更充分地利用信息。当前模型基于静态历史数据训练。为适应物流业务数据的实时性,可探索在线学习或增量学习算法,使模型能够持续从新产生的运单数据中学习演化,保持预测能力的时效性。在现有 Stacking 框架基础上,可以引入动态权重分配机制,让元模型能够根据输入样本的特性动态调整基模型的权重,实现更精细化的预测。

本文构建的“地理聚类分析+集成学习预测”模型框架具有较强的通用性,可推广至物流行业乃至其他领域的多种业务场景。此模型可应用于仓库库位优化。

通过聚类分析高周转率或易损货物的出入库数据，识别“热点”作业区域，从而优化货物摆放布局，提升拣货效率和减少货损；将风险预测模型与路径规划算法结合，在规划运输路线时，不仅考虑距离和时间成本，还将途径区域的 historical 风险概率作为一项成本因子，自动规避高风险路段或站点，实现安全与效率的平衡；还可以基于客户的运单历史数据，利用聚类和分类模型对客户进行分群，识别高价值客户、高风险客户等，为差异化服务策略和精准营销提供数据支持。

八、参考文献

- [1]Wu, P.-J., Chen, M.-C., & Tsau, C.-K. (2017). The data-driven analytics for investigating cargo loss in logistics systems. *International Journal of Physical Distribution & Logistics Management*.
- [2]Liang, X., et al. (2022). Risk analysis of cargo theft from freight supply chains using a data-driven Bayesian network (研究/文章在线摘要).
- [3]Severino, M. K., & Peng, Y. (2021). Machine learning algorithms for fraud prediction in property insurance: Empirical evidence using real-world microdata. *Machine Learning with Applications*.
- [4]马欣. 基于复合事件处理的车险理赔风险防渗漏应用研究[J]. 保险理论与实践, 2020(5): 78-86.
- [5]赵星. 车险赔付率分析与预测中的新技术实践[J]. 保险理论与实践, 2023(9): 45-53.

九、附录

使用工具：SPSSPRO，Python

代码：

第一问：基于 Haversine+K-distance 的 DBSCAN 聚类分析

```
import pandas as pd
from sklearn.cluster import DBSCAN
import numpy as np
# =====
# 1. 数据读取与预处理
# =====
FILE_PATH = r'D:/Mathorcup 复赛/附件 1 处理后.xlsx'
OUTPUT_CSV_PATH = r'D:/Mathorcup 复赛/clustered_datav3.csv'
STATS_CSV_PATH = r'D:/Mathorcup 复赛/cluster_statsv3.csv'
# 最佳参数 V6 (您可以根据需要更改)
EPSILON = 0.001
MIN_SAMPLES = 30

print("--- 开始数据加载和 DBSCAN 聚类 ---")
df = pd.read_excel(FILE_PATH, sheet_name='Sheet1')
# 清洗掉绘图和聚类必须的缺失值
```



```

df = df.dropna(subset=['latitude', 'longitude', 'poi_id', 'claim_flag']).copy()

# =====
# 2. DBSCAN 聚类
# =====
coords = np.deg2rad(df[['latitude', 'longitude']].values) # 转换为弧度
print(f"开始 DBSCAN (Eps={EPSILON}, Min={MIN_SAMPLES})... 数据量: {len(df)}")

db = DBSCAN(eps=EPSILON, min_samples=MIN_SAMPLES, metric='haversine', n_jobs=-1)
df['cluster_id'] = db.fit_predict(coords)

noise_perc = (df['cluster_id'] == -1).sum() / len(df) * 100
print(f"聚类完成。噪声百分比: {noise_perc:.2f}%")
print(f"有效簇数量: {df['cluster_id'].nunique() - (1 if -1 in df['cluster_id'].unique() else 0)}")

# 过滤噪声点，只保留有效簇的数据
df_clustered = df[df['cluster_id'] != -1].copy()
# =====
# 3. 计算簇统计信息 (用于高风险区域判断)
# =====
cluster_stats = df_clustered.groupby('cluster_id').agg(
    total_orders=('waybill_code', 'count'),
    abnormal_count=('claim_flag', 'sum'),
    center_lat=('latitude', 'mean'),
    center_lon=('longitude', 'mean'),
    poi_mode=('poi_id', lambda x: x.mode()[0] if not x.mode().empty else 'Unknown')
).reset_index()

cluster_stats['abnormal_rate'] = cluster_stats['abnormal_count'] / cluster_stats['total_orders']

# 风险区域判断 (例如: 异常率大于 2% 即为风险)
RISK_THRESHOLD = 0.02
cluster_stats['is_risk_area'] = np.where(cluster_stats['abnormal_rate'] > RISK_THRESHOLD, '是', '否')
# =====
# 4. 数据保存
# =====
# A. 保存聚类后的原始数据 (包含经纬度、理赔标志和簇 ID)
df_to_save = df_clustered[['longitude', 'latitude', 'cluster_id', 'claim_flag']].copy()
df_to_save.to_csv(OUTPUT_CSV_PATH, index=False, encoding='utf-8-sig')
print(f"\n 聚类散点数据已保存到: {OUTPUT_CSV_PATH}")

# B. 保存簇的统计信息 (包含异常率等)
cluster_stats.to_csv(STATS_CSV_PATH, index=False, encoding='utf-8-sig')

```

```

print(f"簇统计信息已保存到: {STATS_CSV_PATH}")

print("--- DBSCAN 处理完毕 ---")

import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import Normalize
import numpy as np
print("--- 开始地图加载与可视化 ---")

# =====
# 2. 数据加载
# =====
try:
    df_clustered = pd.read_csv(CLUSTERED_DATA_PATH)
    cluster_stats = pd.read_csv(STATS_CSV_PATH)

    # 合并异常率到散点数据中，用于配色
    rate_map = cluster_stats.set_index('cluster_id')['abnormal_rate'].to_dict()
    df_clustered['abnormal_rate'] = df_clustered['cluster_id'].map(rate_map)

    # 过滤出高风险区域
    RISK_THRESHOLD = 0.02
    high_risk_clusters = cluster_stats[cluster_stats['abnormal_rate'] >
RISK_THRESHOLD]['cluster_id']
    high_risk_df = df_clustered[df_clustered['cluster_id'].isin(high_risk_clusters)].copy()

    china_map = gpd.read_file(MAP_FILE_PATH)
    print("数据和地图文件加载成功！")

except Exception as e:
    print(f"加载文件失败，请检查路径和文件是否存在。错误: {e}")
    # 退出绘图流程
    exit()

# =====
# 3. 绘图函数 (通用逻辑)
# =====
def plot_map_scatter(df_plot, title, hue_column=None, palette='viridis', legend=True,
output_filename='plot.png', is_risk_map=False):
    """
    绘制带中国轮廓的散点图。
    """

```

```

fig, ax = plt.subplots(figsize=(12, 10))

# 绘制地图轮廓
china_map.plot(ax=ax, color='#f5f5f5', edgecolor='gray', linewidth=0.8, zorder=1)

# 绘制散点
scatter = sns.scatterplot(
    x='longitude',
    y='latitude',
    hue=hue_column,
    data=df_plot,
    palette=palette,
    legend=legend,
    s=20,
    ax=ax,
    zorder=2,
    alpha=0.7
)

if is_risk_map and hue_column:
    # 针对高风险图添加颜色条
    norm = plt.Normalize(df_plot[hue_column].min(), df_plot[hue_column].max())
    sm = plt.cm.ScalarMappable(cmap="Reds", norm=norm)
    sm.set_array([])
    cbar = plt.colorbar(sm, ax=ax, label='Abnormal Rate')
plt.title(title)
# 锁定经纬度范围以适应中国地图
plt.xlim(73, 136)
plt.ylim(18, 54)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
# 调整图例
if not legend:
    if scatter.legend_ is not None: # 防止 NoneType 报错
        scatter.legend_.remove()

plt.savefig(output_filename, dpi=300, bbox_inches='tight')
plt.close()
print(f"图表已保存到: {output_filename}")
# =====
# 4. 绘图执行
# =====
# A. 绘制所有簇的散点图
# A. 绘制所有簇的散点图 (地图)

```

```

plot_map_scatter(
    df_clustered,
    title=f'DBSCAN Clusters (model: Eps={EPSILON}, Min={MIN_SAMPLES})',
    hue_column='cluster_id',
    legend=False,
    output_filename=r'D:/Mathorcup 复赛/clusters_visualization_V3.png'
)
# B. 绘制高风险区域图 (地图)
plot_map_scatter(
    high_risk_df,
    title='High Risk Areas Distribution ',
    hue_column='abnormal_rate',
    palette='Reds',
    legend=False,
    output_filename=r'D:/Mathorcup 复赛/high_risk_areas_V3.png',
    is_risk_map=True
)
print("--- 可视化任务完成 ---")

```

第二问：LightGBM + XGBoost + CatBoost 三模型集成方案 (Stacking 版)

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import roc_auc_score, precision_recall_curve, confusion_matrix, roc_curve
from sklearn.linear_model import LogisticRegression
import lightgbm as lgb
import xgboost as xgb
from catboost import CatBoostClassifier, Pool
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import os

warnings.filterwarnings('ignore')
# =====
# 0. 全局配置
# =====
TRAIN_PATH = 'D:/Mathorcup 复赛/附件 1raw.xlsx'
TEST_PATH = 'D:/Mathorcup 复赛/附件 2raw.xlsx'
OUTPUT_FILE = 'D:/Mathorcup 复赛/结果表 2_集成模型.xlsx'
TARGET_TOP_N = 300
N_FOLDS = 5

```

```

# =====
# 1. 数据加载与预处理
# =====
def load_and_preprocess():
    print("正在加载数据...")
    train_df = pd.read_excel(TRAIN_PATH)
    test_df = pd.read_excel(TEST_PATH)

    # 保存测试集运单号用于最终提交
    test_waybill_codes = test_df['waybill_code'].copy()

    # 移除无用的 ID 列
    drop_cols = ['waybill_code']
    train_df = train_df.drop(columns=drop_cols, errors='ignore')
    test_df = test_df.drop(columns=drop_cols, errors='ignore')

    print(f"数据加载完成 - 训练集: {train_df.shape}, 测试集: {test_df.shape}")
    return train_df, test_df, test_waybill_codes
# =====
# 2. 特征工程 (差异化处理)
# =====
def feature_engineering(train_df, test_df):
    """
    构建通用特征，并为不同模型准备数据格式。
    CatBoost: 使用原始类别特征 (字符串/整数)
    LGBM/XGB: 使用编码后的类别特征 (数值)
    """
    print("开始特征工程...")

    # 1. 基础填充缺失值
    num_cols = ['insured_value', 'delivery_waybill_count', 'pickup_waybill_count']
    for col in num_cols:
        fill_val = train_df[col].mean()
        train_df[col] = train_df[col].fillna(fill_val)
        test_df[col] = test_df[col].fillna(fill_val)

    # 类别列填充 'unknown' 或 -1
    cat_cols = ['delivery_site_id', 'pickup_site_id', 'product_type', 'transport_type',
                'resend_flag', 'private_waybill_flag', 'protect_flag', 'insurance_flag',
                'damage_flag', 'photo_flag', 'town_id', 'goods_type', 'poi_id']

    for col in cat_cols:
        train_df[col] = train_df[col].fillna('unknown').astype(str)
        test_df[col] = test_df[col].fillna('unknown').astype(str)

```

```

# 2. 经纬度处理 (分箱 + 缺失标记)
for col in ['latitude', 'longitude']:
    train_df[col] = train_df[col].fillna(-999.0)
    test_df[col] = test_df[col].fillna(-999.0)

# 创建缺失标记
train_df['lat_missing'] = (train_df['latitude'] == -999.0).astype(int)
test_df['lat_missing'] = (test_df['latitude'] == -999.0).astype(int)
n_bins = 20
for col in ['latitude', 'longitude']:
    _, bins = pd.cut(train_df[col], bins=n_bins, retbins=True, duplicates='drop')
    train_df[f'{col}_bin'] = pd.cut(train_df[col], bins=bins, labels=False,
include_lowest=True).fillna(-1).astype(int)
    test_df[f'{col}_bin'] = pd.cut(test_df[col], bins=bins, labels=False,
include_lowest=True).fillna(-1).astype(int)

# 3. 构建衍生特征
epsilon = 1e-5
# 站点单量比率 (发件/收件)
train_df['site_ratio'] = train_df['pickup_waybill_count'] / (train_df['delivery_waybill_count']
+ epsilon)
test_df['site_ratio'] = test_df['pickup_waybill_count'] / (test_df['delivery_waybill_count'] +
epsilon)

# --- 数据集拆分 ---

# A. 为 CatBoost 准备的数据 (保留 cat_cols 为字符串)
# 只需要确保 cat_cols 在列表中, 且对应列是字符串/整数即可
X_train_cat = train_df.drop(columns=['claim_flag'])
X_test_cat = test_df.copy()

# B. 为 LGBM / XGBoost 准备的数据 (类别特征 LabelEncoding)
X_train_num = train_df.drop(columns=['claim_flag']).copy()
X_test_num = test_df.copy()

le_dict = {}
for col in cat_cols:
    le = LabelEncoder()
    # 联合编码防止从没见过的类别报错
    full_data = pd.concat([X_train_num[col], X_test_num[col]], axis=0).astype(str)
    le.fit(full_data)
    X_train_num[col] = le.transform(X_train_num[col].astype(str))
    X_test_num[col] = le.transform(X_test_num[col].astype(str))

```

```

le_dict[col] = le

y_train = train_df['claim_flag']

return X_train_cat, X_test_cat, X_train_num, X_test_num, y_train, cat_cols

# =====
# 3. 模型训练函数 (为 Stacking 准备 OOF 预测)
# =====

def get_oof_and_test_preds(model_func, X_num, X_cat, y, X_test_num, X_test_cat, cat_cols,
model_name):
    kf = KFold(n_splits=N_FOLDS, shuffle=True, random_state=42)
    oof_preds = np.zeros(len(y))
    test_preds = np.zeros((len(X_test_num), N_FOLDS))

    for fold, (train_idx, val_idx) in enumerate(kf.split(X_num)):
        print(f"\n[{model_name}] Fold {fold+1}/{N_FOLDS}")

        # 数值数据拆分
        X_tr_num, X_val_num = X_num.iloc[train_idx], X_num.iloc[val_idx]
        y_tr, y_val = y.iloc[train_idx], y.iloc[val_idx]

        # 类别数据拆分
        X_tr_cat, X_val_cat = X_cat.iloc[train_idx], X_cat.iloc[val_idx]

        # 训练并获取预测
        if model_name in ['LGB', 'XGB']:
            val_pred, test_pred_fold, _ = model_func(X_tr_num, X_val_num, y_tr, y_val,
X_test_num)
        else: # CatBoost
            val_pred, test_pred_fold, _ = model_func(X_tr_cat, X_val_cat, y_tr, y_val,
X_test_cat, cat_cols)

        oof_preds[val_idx] = val_pred
        test_preds[:, fold] = test_pred_fold

    # 测试集预测平均
    test_preds_avg = test_preds.mean(axis=1)

    # OOF AUC
    oof_auc = roc_auc_score(y, oof_preds)
    print(f"[{model_name}] OOF AUC: {oof_auc:.4f}")

```

```

    return oof_preds, test_preds_avg

def train_lgb_model(X_train, X_val, y_train, y_val, X_test):
    print("\n 训练 LightGBM...")

    # 计算权重
    scale_pos_weight = (len(y_train) - sum(y_train)) / sum(y_train) if sum(y_train) > 0 else 1

    params = {
        'objective': 'binary',
        'metric': 'auc',
        'boosting_type': 'gbdt',
        'learning_rate': 0.03,
        'num_leaves': 31,
        'max_depth': 7,
        'scale_pos_weight': scale_pos_weight,
        'feature_fraction': 0.8,
        'bagging_fraction': 0.8,
        'bagging_freq': 5,
        'verbose': -1,
        'seed': 42
    }

    train_data = lgb.Dataset(X_train, label=y_train)
    val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)

    model = lgb.train(
        params,
        train_data,
        valid_sets=[val_data],
        num_boost_round=1000
    )

    # 预测概率
    val_pred = model.predict(X_val)
    test_pred = model.predict(X_test)
    val_auc = roc_auc_score(y_val, val_pred)

    print(f"LightGBM 验证集 AUC: {val_auc:.4f}")
    return val_pred, test_pred, val_auc

def train_xgb_model(X_train, X_val, y_train, y_val, X_test):
    print("\n 训练 XGBoost...")

```



```

scale_pos_weight = (len(y_train) - sum(y_train)) / sum(y_train) if sum(y_train) > 0 else 1

params = {
    'objective': 'binary:logistic',
    'eval_metric': 'auc',
    'learning_rate': 0.03,
    'max_depth': 6,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'scale_pos_weight': scale_pos_weight,
    'seed': 42,
    'n_jobs': -1
}

dtrain = xgb.DMatrix(X_train, label=y_train)
dval = xgb.DMatrix(X_val, label=y_val)
dtest = xgb.DMatrix(X_test)

model = xgb.train(
    params,
    dtrain,
    num_boost_round=1000,
    evals=[(dtrain, 'train'), (dval, 'eval')],
    early_stopping_rounds=50,
    verbose_eval=100
)

val_pred = model.predict(dval)
test_pred = model.predict(dtest)
val_auc = roc_auc_score(y_val, val_pred)

print(f"XGBoost 验证集 AUC: {val_auc:.4f}")
return val_pred, test_pred, val_auc

def train_cat_model(X_train, X_val, y_train, y_val, X_test, cat_cols):
    print("\n 训练 CatBoost (修复报错版)...")

    scale_pos_weight = (len(y_train) - sum(y_train)) / sum(y_train) if sum(y_train) > 0 else 1

    # 确保 cat_cols 在 DataFrame 中存在且为字符串类型
    # 双重检查: 移除可能存在的浮点数列 (如 latitude)
    safe_cat_cols = [c for c in cat_cols if c in X_train.columns]

    # 打印检查

```

```

print(f'CatBoost 使用的分类特征: {safe_cat_cols}')

params = {
    'iterations': 1000,
    'learning_rate': 0.03,
    'depth': 7,
    'loss_function': 'Logloss',
    'eval_metric': 'AUC',
    'scale_pos_weight': scale_pos_weight,
    'cat_features': safe_cat_cols,
    'early_stopping_rounds': 50,
    'verbose': 100,
    'random_seed': 42,
    'allow_writing_files': False
}

# 使用 Pool 对象可以更稳定地处理特征类型
train_pool = Pool(X_train, y_train, cat_features=safe_cat_cols)
val_pool = Pool(X_val, y_val, cat_features=safe_cat_cols)
test_pool = Pool(X_test, cat_features=safe_cat_cols)

model = CatBoostClassifier(**params)
model.fit(train_pool, eval_set=val_pool)

val_pred = model.predict_proba(val_pool)[:, 1]
test_pred = model.predict_proba(test_pool)[:, 1]
val_auc = roc_auc_score(y_val, val_pred)

print(f'CatBoost 验证集 AUC: {val_auc:.4f}')
return val_pred, test_pred, val_auc
# =====
# 4. 模型评估与可视化
# =====
def evaluate_and_visualize(y, preds_dict, ensemble_val_pred, cat_model):
    print("\n 开始模型评估与可视化...")

    # 1. ROC 曲线
    plt.figure(figsize=(10, 8))
    for name, pred in preds_dict.items():
        fpr, tpr, _ = roc_curve(y, pred)
        plt.plot(fpr, tpr, label=f'{name} AUC={roc_auc_score(y, pred):.4f}')
    fpr, tpr, _ = roc_curve(y, ensemble_val_pred)
    plt.plot(fpr, tpr, label=f'Ensemble AUC={roc_auc_score(y, ensemble_val_pred):.4f}',
linewidth=3)

```

```

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend()
plt.savefig('roc_curves.png')
plt.close()

# 2. PR 曲线 (集成模型)
precision, recall, _ = precision_recall_curve(y, ensemble_val_pred)
plt.figure(figsize=(8, 6))
plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Ensemble Precision-Recall Curve')
plt.savefig('pr_curve.png')
plt.close()

# 3. Confusion Matrix (集成模型, 阈值 0.5)
cm = confusion_matrix(y, (ensemble_val_pred > 0.5).astype(int))
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Ensemble Confusion Matrix')
plt.savefig('confusion_matrix.png')
plt.close()

# 4. Feature Importance
if cat_model is not None:
    feat_imp = pd.Series(cat_model.get_feature_importance(),
index=cat_model.feature_names_.sort_values(ascending=False)
    plt.figure(figsize=(12, 8))
    feat_imp.plot(kind='bar')
    plt.title('CatBoost Feature Importance')
    plt.savefig('feature_importance.png')
    plt.close()

# =====
# 5. 主函数
# =====
def main():
    # 1. 加载
    train_df, test_df, test_waybill_codes = load_and_preprocess()

    # 2. 特征工程

```

```

# 得到两套数据: cat 用于 CatBoost, num 用于 LGBM/XGB
X_train_cat, X_test_cat, X_train_num, X_test_num, y, cat_cols =
feature_engineering(train_df, test_df)

# 3. 获取基模型 OOF 和测试预测
print("\n[1/3] 训练 LightGBM (OOF)...")
lgb_oof, lgb_test = get_oof_and_test_preds(train_lgb_model, X_train_num, X_train_cat, y,
X_test_num, X_test_cat, cat_cols, 'LGB')

print("\n[2/3] 训练 XGBoost (OOF)...")
xgb_oof, xgb_test = get_oof_and_test_preds(train_xgb_model, X_train_num, X_train_cat, y,
X_test_num, X_test_cat, cat_cols, 'XGB')
print("\n[3/3] 训练 CatBoost (OOF)...")
cat_oof, cat_test = get_oof_and_test_preds(train_cat_model, X_train_num, X_train_cat, y,
X_test_num, X_test_cat, cat_cols, 'CAT')

# 4. Stacking: 使用 OOF 作为元训练特征, 测试平均作为元测试特征
print("\n[4/4] Stacking 融合...")
meta_train = np.column_stack((lgb_oof, xgb_oof, cat_oof))
meta_test = np.column_stack((lgb_test, xgb_test, cat_test))

# 元模型: LogisticRegression
meta_model = LogisticRegression(random_state=42)
meta_model.fit(meta_train, y)

# 元模型预测
ensemble_val_pred = meta_model.predict_proba(meta_train)[:, 1]
ensemble_test_pred = meta_model.predict_proba(meta_test)[:, 1]

ensemble_auc = roc_auc_score(y, ensemble_val_pred)
print(f"Stacking 集成模型 OOF AUC: {ensemble_auc:.4f}")

# 5. 模型评估与可视化
preds_dict = {
    'LGB': lgb_oof,
    'XGB': xgb_oof,
    'Cat': cat_oof
}

idx_train, idx_val = train_test_split(np.arange(len(y)), test_size=0.2, stratify=y,
random_state=42)
X_tr_cat, X_val_cat = X_train_cat.iloc[idx_train], X_train_cat.iloc[idx_val]
y_tr, y_val = y.iloc[idx_train], y.iloc[idx_val]
_, _, _ = train_cat_model(X_tr_cat, X_val_cat, y_tr, y_val, X_test_cat, cat_cols)
scale_pos_weight = (len(y) - sum(y)) / sum(y) if sum(y) > 0 else 1
safe_cat_cols = [c for c in cat_cols if c in X_train_cat.columns]

```

```

params = {
    'iterations': 1000,
    'learning_rate': 0.03,
    'depth': 7,
    'loss_function': 'Logloss',
    'eval_metric': 'AUC',
    'scale_pos_weight': scale_pos_weight,
    'cat_features': safe_cat_cols,
    'early_stopping_rounds': 50,
    'verbose': 100,
    'random_seed': 42,
    'allow_writing_files': False
}
train_pool = Pool(X_train_cat, y, cat_features=safe_cat_cols)
cat_model = CatBoostClassifier(**params)
cat_model.fit(train_pool)
evaluate_and_visualize(y, preds_dict, ensemble_val_pred, cat_model)
# 6. 生成 Top 300 结果
print(f"\n 正在生成 Top {TARGET_TOP_N} 推荐结果...")
result_df = pd.DataFrame({
    'waybill_code': test_waybill_codes,
    'claim_prob': ensemble_test_pred
})
result_df = result_df.sort_values(by='claim_prob', ascending=False)
# 选取 Top N
top_n_df = result_df.head(TARGET_TOP_N).copy()
result_df['recommend_service'] = 0
result_df.iloc[:TARGET_TOP_N, result_df.columns.get_loc('recommend_service')] = 1

# 保存结果
result_df.to_excel(OUTPUT_FILE, index=False)
print(f"结果已保存至: {OUTPUT_FILE}")
print(f"推荐服务运单数: {result_df['recommend_service'].sum()}")
print("前 5 个高风险运单:")
print(result_df.head())

if __name__ == '__main__':
    main()

```